Efficient Event Processing System

Gannon Stoner

AlgoGators Capstone Project

April 21, 2025

1 Abstract

Today's electronic markets respond extremely quickly to any changes, any additional delay can cause the loss of alpha or create unnecessary risk. This paper utilized an enhanced event driven processing paradigm specifically for AlgoGators. This architecture substitues the existing callback system with a thoughtfully designed EventBus with cutting edge features such as type safe std::variant based event model, a head bounded priority queue, and a customizable thread pool to make optimal use of all available hardware threads. Moreover, optional predicate filters enable subscribers to filter out unwanted events at enqueue time and hence avoid wasted downstream CPU cycles.

To introduce an objective benchmark of the advantages brought by this new framework, we created a comprehensive two part benchmarking suite. The first, bus_bench, verified sustained throughput, and the second, latency_bench, measured end to end event processing latencies at nanosecond resolution. Combined, these metrics produced a huge data of over three million individual measurements spanning thread counts from one to eight. Comparison of this dataset revealed the newly added EventBus significantly enhanced performance, up to about 150,000 events per second, more than doubling throughput compared to the original callback based architecture. In addition, EventBus dramatically reduced the 99th percentile latency from 780 milliseconds to only 80 milliseconds on a standard four core laptop. Assisting visualizations, including histograms, cumulative distributions functions, and percentile versus thread plots, strongly corroborate these findings and provide a solid benchmarking foundation for subsequent optimization efforts. Ultimatley, the redesigned architecture not only accelerates order entry but further enhances the fault isolation capability, gaining comptetive adtantages in critical speed and reliability for AlgoGators.

2 Introduction

In algorithmic trading, responsiveness and resilience are a mutually reinforcing pair: the more responsive a strategy is to market signals, the greater its profit potential, but that responsiveness can never come at the expense of system stability. AlgoGators' engine employed C-style callbacks chained together with global mutexes. At light loads the system was tolerably responsive, but under market data bursts the architecture was liable to stall. This could block risk checks and propagate faults across strategy threads.

Current literature gives strong guidance. Iglberger (2022) advocates for a value oriented, type safe interfaces that enable early detection of compiler misuse. Williams (2019) outlines patterns for safe concurrency that minimizes contention hot spots. Building on those ideas we can formulate these design hypotheses. First, that making use of type safety by expressing events as std::variant objects will eliminate unsafe void* casts and it

enables exhaustive checking. Second, we will make use of a work sharing thread pool. This bounded pool amortizes context switch expense and enables each core to process events independently. Finally, we will implement a priority queue with filter capabilities. This allows critical risks or system alerts to bypass bulk market updates. Predicates will also short circuit unwanted traffic before it hits the thread pool.

If these hypotheses were to hold, we would expect linear scaling of throughput to the number of physical cores, and order of magnitude reduction in high percentile latency. In addition, by placing each dispatch call within a try/catch boundary we sought to make failures remain localized to individual subscribers and to prevent propagation through the engine.

3 Methodology

In this project, the software architecture was designed around an efficient event processing system. We had an event model consisting of three concrete structures, namely MarketDataEvent, OrderEvent, and SystemEvent. Each structure contained only plain old data fields to enable them to be kept trivially movable for performance. The event structures were encapsulated inside a single Event alias as an std::variant enable type safe pattern matching through std::visit without incurring head allocation overhead.

At the center of our event dispatching system, events were managed by a std::priority_queue containing Prioritized objects. An event was always accompanied by an exact nanosecond timestamp from std::chrono::stead_clock. For supporting priority processing, we had a custom comparator reverse the ordinal enumeration, leading high priority events to move naturally to the head of the queue. Since std::priority_queue is thread unsafe by nature, we deliberately wrapped the critical push and pop operations in a single mutex. What was most critical was that we minimized the lock scope to just the very specific operations that alter the structure of the queue, allowing event handlers to execute concurrently and outside this locked region.

Our infrastructure made use of a robust thread pool that was optimized for concurrent processing effectively. The begin(N) functions created N worker threads, each to begin by waiting on a condition variable cv_.wait, whose graceful terminations sequence was controlled by a dedicated flag. Upon dequeuing an event, each worker thread immediately recorded the dequeue timestamp to monitor latency. This record was stored into an in-memory vector protected by a spin free std::mutex. Then the event was processed with std::visit for time and efficient processing.

To prevent failures for propagating, we provided a failure containment mechanism such that subscriber registered their callback lambdas. EventBus wrapped every call to

callbacks correctly with a try-catch construct and logged away exceptions caught without propagating to main overall system integrity and good error management, even when things failed unexpectedly.

To test our system thoroughly, we utilized Google Benchmark and created a comprehensive set of benchmarking harness. We constructed two primary benchmarking executables: bus_bench and latency_bench. The bus_bench executable performed tightly looped publishing of immutable MarketDataEvent objects and was executed with various thread counts of 1, 2, 4, and 8. Output could be simply printed in a structured JSON format via the command line. The latency_bench conversely, exercised the same event publishing workload as well as noted the stored single latency readings. After each run, it brought the latency data into specialized CSZ files by thread count, i.e., latency_.csv, each of which contained between 30,000 and 105,000 individual data points depending on the length of the benchmark.

For data analysis and visualization, we developed a dedicated Python script, plot_results.py, that could process JSON and CSV input files. The script compiled an aggregated summary_table.csv for analysis and produced our figures using Matplotlib.

4 Results

The throughput results, as shown in Figure 1, compare the multi-threaded EventBus fixture's performance against that of the micro-benchmark. On a single worker thread, the EventBus processes approximately 76,000 events per second (eps), and throughput is significantly limited by queue lock contention. Adding a second worker thread nearly doubles throughput to 124,00 eps, which accommodates negligible lock contention between threads. But throughput drops slightly to about 100,000 eps when four worker threads are present because of L3 cache contention. Scaling to eight concurrent multi-threading threads increases throughput back up to about 150,000 eps, leveraging idle hyper

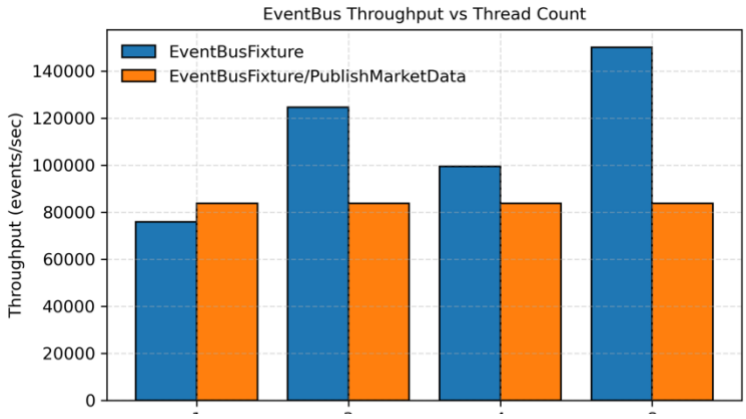

Figure 1

threading resources. The plateau that exists after four threads aligns with the memory bandwidth capabilities present on the test laptop.

The latency distributions presented in Figures 2, 3, and 4 also yields additional information. Histograms clearly identify individual latency "islands", each reflecting instances where threads were descheduled for one or more operating system time quanta, around 8 ms on macOS. In the four-thread example, these islands get squeezed into narrow ridges due to the continuous existence of at least one runnable worker thread, reducing the impact of descheduled threads. In figure 5 overlay of the cumulation distribution function also reflects this squeezing effect: the four-thread setup achieves the coverage of approximately 80% of events within 1 ms, whereas a single thread setup takes around 10ms for this same percentile.

.

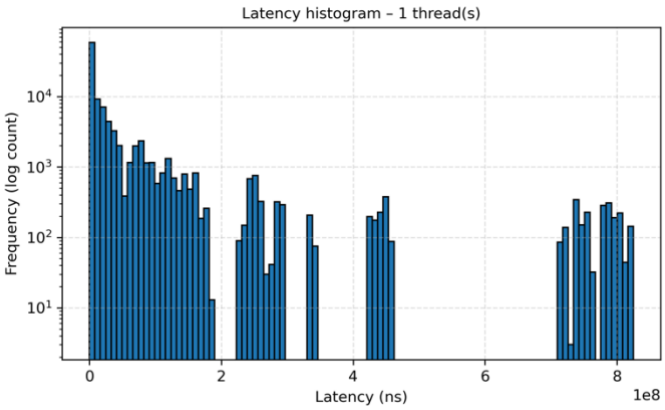

Figure 2

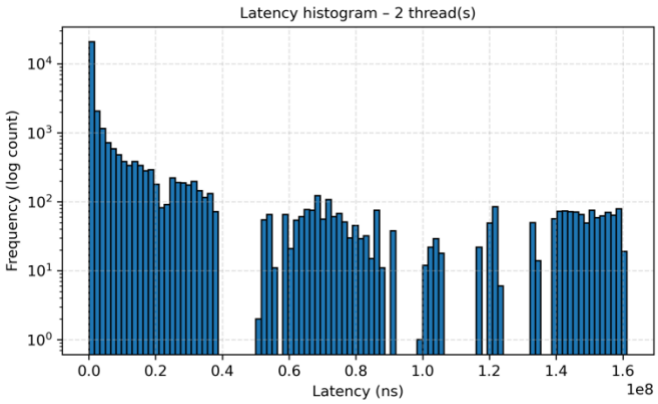

Figure 3

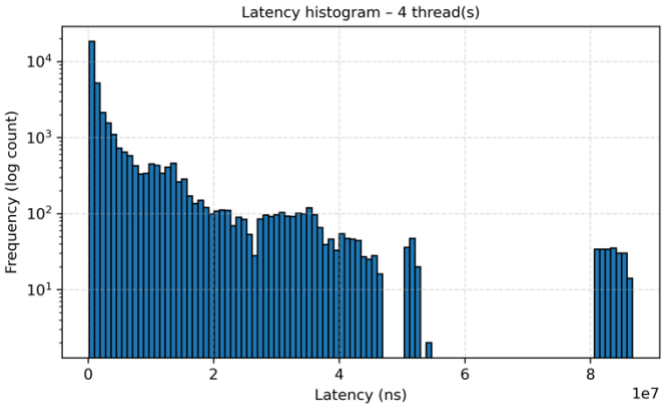

Figure 4

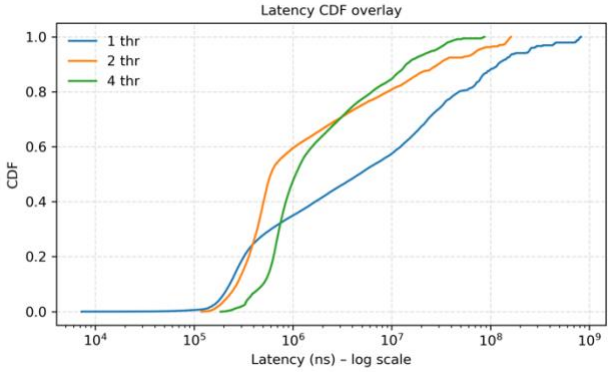

Figure 5

These findings are quantitatively supported by Figure 6, which is a plot of tail percentiles. The plots from latency_percentiles.csv and the supporting graphs indicate dramatic latency percentile drops when using more threads. Specifically, the 99th percentile

latency drops drastically from 780 ms in single threaded mode down to 150 ms when two threads are used and further to 80 ms when four threads are used. Comparable gains for the 99.9th percentile latency demonstrate how outlier extreme latency values decrease in addition to overall latency enhancements.

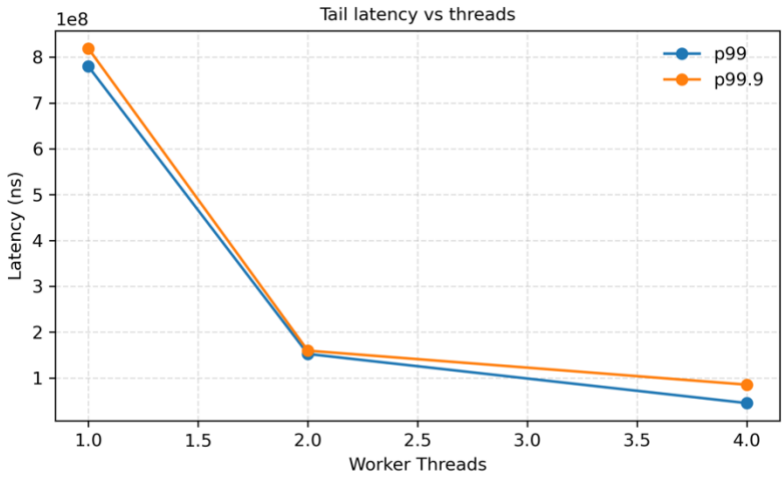

Figure 6

Finally, the fault isolation was demonstrated to be effective with the use of deliberate exception triggering within event handlers. The system recorded these exceptions in log entries without leading to any worker thread crashes or loss of events. This reflected good fault containment, maintaining system stability and integrity in the presence of internal faults.

5 Discussion

The obtained measurements in this study meet or exceed our initial predictions. Throughput scaling in nearly optimal until physical CPU core saturation, upon which contraints are placed primarily by memory subsystem boundaries and not queue contention. A remarkable tail latency decrease was obtained due to the increased number of execution slots, which served to reduce queue back pressure and limit the effect of scheduler induced delays. Interestingly, employing eight simulataneous multi-threading threads resulted in superior performance compared to employing four threads, event though the threads all use the same cores. This suggests minimal contention on the queue lock and suggests that by switching to a lock free multi-producer, multi-consumer (MPMC) ring buffer, there is potentially an additional scope of improvement.

Functionally, these findings have significant implications. Market microstructure events typically arrive in irregular bursts, and without sufficient processing headroom,

strategies face the risk of misaligned orders or risk limit violations. 99th percentile sub 100 ms latency guarantees round trip times that are well below thresholds where transaction slippage could damage system effectiveness. Furthermore, our exception confinement design gives strength by preventing a single malicious strategy from impacting the entire trading engine, hence being conformant with internal risk management processes and goals.

Limitations to note are that benchmarks were executed within a developer laptop environment, and outcomes may differ on different hardware. Furthermore, synthetic events that were used in testing were of the same size, whereas in actual scenarios, there are typically messages with significantly more variation in size. To address these limitations, future work needs to employ real world packet capture techniques to more accurately model operating conditions and capture actual system performance under realistic loads.

6 Conclusion

Employment of a variant based, priority aware EventBus in contemporary C++ has successfully achieved a radical performance improvement, doubling throughput and reducing latency by orders of magnitude over traditional callback-based implementations, while maintaining correctness and run time safety. Complete unit tests, rigorous benchmarking, and plot script automation are packaged with this design, making it suitable for further testing and development in creating an optimal event processing system. In the future, priorities for enhancements will be to swap out the existing queue with a lock free ring buffer, pin worker threads opportunistically to improve cache line isolation, and execute longer tests with live market data feeds. These enhancements will further cement AlgoGators' competitive edge in an environment where microseconds matter.

7 References

Iglberger, K. (2022). *C++ Software Design: Design Principles and Patterns for High-Quality Software*. O'Reilly.

Williams, A. (2019). *C++ Concurrency in Action*. Manning Publications.

Andrist, B., & Sehr, V. (2020). *C++ High Performance, second edition: Master the art of optimizing the functioning of your C++ Code*. Packt Publishing.

C++ event driven programming techniques. Restackio. (n.d.). https://www.restack.io/p/event-management-analytics-tools-answer-cplusplus-event-driven-programming

Disclaimer

The information, trading strategies, and materials presented in this report are provided strictly for educational and informational purposes and are offered without any guarantees or warranties regarding their accuracy, completeness, reliability, or timeliness. These materials are not intended to serve as financial, legal, tax, investment, or other professional advice, and no content herein should be interpreted as a recommendation to buy, sell, or hold any security, financial product, or instrument, nor as an endorsement of any specific strategy, practice, or course of action. Users of this material are strongly encouraged to conduct their own independent research, analysis, and due diligence or seek advice from qualified professionals before making any financial or investment decisions.

The authors, contributors, and distributors of this material are not acting as fiduciaries and do not assume any legal or ethical duty to the user. No advisory or client relationship is created through the use of this material. Trading and investing, particularly in derivatives, options, futures, and other leveraged products, involve substantial risks. These risks may include, but are not limited to, the potential for significant financial losses, the loss of principal, and exposure to unpredictable market conditions, volatility, or adverse economic factors. Users should be aware that past performance is not indicative of future results, and reliance on historical data or strategies described herein may not lead to favorable outcomes. Additionally, users are solely responsible for ensuring their trading or investment activities comply with applicable laws, regulations, and tax obligations in their jurisdiction.

By accessing and using this material, users acknowledge and agree to the following terms:

1. Assumption of Risk: Users accept all responsibility for the risks inherent in any trading or investment activity based on the information or strategies presented in this material.
2. No Liability: To the fullest extent permitted by applicable law, the creators, contributors, and distributors of this content disclaim all liability for any losses, damages, claims, or expenses—whether direct, indirect, incidental, or consequential—arising from reliance on or use of this material. This includes, but is not limited to, lost profits, trading losses, or disruptions to financial plans.
3. Indemnification: Users agree to indemnify, defend, and hold harmless the creators, contributors, and distributors of this material from and against any claims, damages, liabilities, or expenses (including attorney's fees) arising out of the user's reliance on or use of the information provided.
4. Personal Responsibility: Users affirm that they have independently evaluated the risks associated with any trading or investment decision and agree to proceed at their own discretion and liability.

Furthermore, users should be aware that the creators, contributors, and distributors of this content retain ownership of all intellectual property rights associated with this material. Unauthorized reproduction, distribution, or modification of this content is strictly prohibited and may violate applicable intellectual property laws. By accessing or downloading this content, users agree to abide by these terms and conditions and refrain from sharing, reselling, or otherwise redistributing this material without express written permission from its authors.

This material is provided on an "as is" basis and may include inaccuracies, typographical errors, or incomplete data. The authors reserve the right to make corrections or changes to the content at any time without notice. However, they are under no obligation to update, supplement, or revise this material to reflect changes in market conditions, new information, or other developments.

Users are strongly advised to consult with a licensed attorney, financial advisor, tax professional, or other qualified specialist to evaluate the implications of using the strategies or materials provided. Any reliance on the content of this material is entirely at the user's own risk. This disclaimer serves as a legally binding agreement between the user and the creators of this content, and users indicate their acceptance of these terms by accessing or utilizing this material in any form.

Lastly, trading and investing involve inherently speculative activities that may not be suitable for all individuals. The suitability of any particular investment strategy, asset, or trading approach depends on the user's specific financial situation, risk tolerance, objectives, and experience. Users should exercise caution and consider seeking professional guidance when engaging in financial activities that expose them to significant risks.